



KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Constraint Programming

Constraint Propagation

Week 2

Constraint Propagation

Assoc. Prof. Tolga Berber

KTU Faculty of Science, Department of Computer Science · <https://avesis.ktu.edu.tr/tberber>

Learning phases: **1 • Pre-instruction** **2 • Content** **3 • Participation** **4 • Assessment**
5 • Follow-through

1 • Pre-instruction Why It Matters?

A course-registration system lists a time slot as “still available” right up until a student tries to select it and is told every course that could use it is already full.

- Each field's own rule looked fine, checked in isolation
- Nobody had checked whether the **combination** the student was heading toward could still work
- That gap – “looks fine alone” vs. “actually works together” – is exactly what a solver checks before it ever guesses a single value

Last week you called `model.AddAllDifferent(...)` without asking why it helped. This week opens that box.

This Week

🎯 Learning Objectives

By the end of this week, you will be able to:

- 1 Define node, arc and GAC, and hand-check a small CSP at each level
- 2 Trace AC-3 by hand; explain why it terminates and its complexity
- 3 Explain what `AllDifferent`, `Element`, `Table`, `Circuit` and `Cumulative` each propagate
- 4 Build two CP-SAT models of one problem; compare propagation strength via solver statistics
- 5 Connect an observed propagation gap back to the consistency-level argument that predicts it

SECTION 1

Unit 1 — Local Consistency

Local Consistency AC-3 Global Constraints Measuring Strength Summary

2 • Content Node Consistency: Checking a Variable Against Itself

Definition: Node consistency

A variable is node-consistent if every value in its domain satisfies every *unary* constraint on that variable alone.

- Variable `Slot`, domain {9am, 11am, 2pm}, unary rule “not 2pm” (the room is booked)
- Checking `Slot` against its own rule removes 2pm, leaving {9am, 11am}
- No other variable was ever consulted – the cheapest possible filtering step, usually applied first

Arc Consistency: Checking a Value's Support

Definition: (Binary) arc consistency

An arc (X_i, X_j) is consistent if every value remaining in X_i 's domain has a *supporting* value in X_j 's current domain; a CSP is arc-consistent if every arc is.

Analogy

Two students cross-check a shared exam-room sign-up sheet: each asks “for every slot still on your list, is there still a slot on the other list that does not clash with it?” A slot with no such partner is crossed off – exactly what *Revise* removes (Unit 2).

Generalized Arc Consistency: The Same Idea, Any Arity

Definition: Generalized arc consistency (GAC)

A value for X_i is GAC-consistent with a constraint C if it extends to a complete assignment of *every* variable in C 's scope satisfying C .

- Toy: A, B, C , domain $\{1, 2\}$ each, `AllDifferent`(A, B, C) – every value looks fine *pairwise*
- Pigeonhole: no assignment makes all three pairwise-different with two values – GAC catches this; pairwise arcs never do

node $\subset$ **arc** $\subset$ **GAC**: each step asks more of the rest of the problem before trusting a value.

3 • Participation Quick Check: Is This GAC?

Exam slots P, Q, R , each originally domain $\{1, 2, 3\}$, under $\text{AllDifferent}(P, Q, R)$. Slot 3 was cancelled: every domain is now $\{1, 2\}$.

- Is this CSP generalized-arc-consistent?
- Check: for each value of each variable, does it have a support that also respects AllDifferent for the other two?

No – Unit 1's pigeonhole toy with new names: 3 variables, 2 values, no assignment can make all three pairwise-different, so a GAC-aware check empties every domain immediately, while checking $P \neq Q$, $P \neq R$, $Q \neq R$ one pair at a time never finds a wipeout.

SECTION 2

Unit 2 — AC-3 and the Fixpoint

Local Consistency **AC-3** Global Constraints Measuring Strength Summary

2 • Content The AC-3 Algorithm

- Maintain a **worklist** of arcs still to check
- Pop an arc (X_i, X_j) , run **Revise** (X_i, X_j) : remove every value of X_i with no support in X_j
- If **Revise** changed X_i , **re-add every arc** (X_k, X_i) for a neighbour X_k – a value that supported X_k may have just vanished from X_i
- Stop when the worklist is empty

Analogy

A lab's shared equipment sign-up sheet: crossing a name off one calendar because a slot fell through forces you to re-check every *other* calendar that shared that slot – you keep re-checking until nobody's calendar changes anymore.

Worked by Hand: The Chain $X < Y < Z$

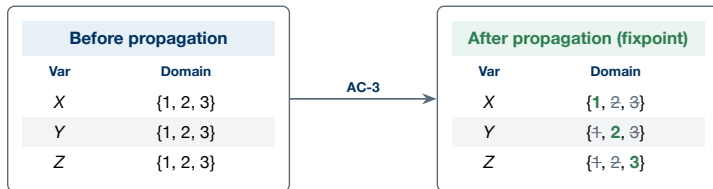
Domains $\{1, 2, 3\}$ each, constraints $X < Y, Y < Z$.

- Worklist starts as $(X, Y), (Y, X), (Y, Z), (Z, Y)$
- (X, Y) : removes $X=3$ (no $y > 3$); X has no other neighbour, nothing re-queued
- (Y, X) : removes $Y=1$ (no $x < 1$); Y changed, so (Z, Y) is re-queued
- (Y, Z) : removes $Y=3$ (no $z > 3$), $Y=\{2\}$; Y changed again, so (X, Y) is re-queued
- (Z, Y) : removes $Z=1, Z=2$ against $Y=\{2\}$, $Z=\{3\}$; the duplicate (Z, Y) then finds nothing to do
- (X, Y) again, against $Y=\{2\}$: removes $X=2$, $X=\{1\}$

Fixpoint: $X=\{1\}, Y=\{2\}, Z=\{3\}$ – for this chain, arc consistency alone reached the unique solution, no search needed.

Before and After: The Fixpoint at a Glance

Domain propagation on the chain CSP $X < Y < Z$



Chain CSP $X < Y < Z$: arc consistency alone reaches the unique solution — see Week 5 for when this always happens.

Every removed value is the direct result of one **Revise** call from the trace on the previous slide.

Termination, Complexity, and the Name “Fixpoint”

- AC-3 always terminates: each **Revise** either removes at least one value or changes nothing, and no domain can shrink past empty – the worklist cannot refill forever
- Classic bound for binary CSPs: $O(e \cdot d^3)$, e arcs, d the largest domain size (Apt / Dechter)
- **Fixpoint**: the state where no further **Revise** call changes anything – Weeks 7 and 10 reuse this exact word for LP pivoting and dual updates

Code: The Revise Step

```
1 changed = False
2 for value_i in set(domains[xi]):
3     if not any(constraint(value_i, value_j) for value_j in domains[xj]):
4         domains[xi].remove(value_i)
5         changed = True
6 return changed
```

Every line here is one clause of the earlier definition, written down directly: loop over X_i 's values, drop any with no support in X_j .

Code: The Worklist Loop

```
1 worklist = deque(arcs)
2 while worklist:
3     xi, xj = worklist.popleft()
4     if revise(domains, xi, xj, constraints[(xi, xj)]):
5         if not domains[xi]:
6             return False
7         for xk in neighbours[xi]:
8             if xk != xj:
9                 worklist.append((xk, xi))
10 return True
```

The re-queueing step (`worklist.append((xk, xi))`) is exactly the “re-check every calendar that shared that slot” step from the equipment sign-up analogy.

Real Output: The Chain CSP, Independently Checked

```
1 print(f"Domains before AC-3: {format_domains(domains)}")
2 consistent = ac3(domains, arcs, constraints, neighbours)
3 print(f"Arc-consistent: {consistent}")
4 print(f"Domains after AC-3 (fixpoint): {format_domains(domains)}")
```

Expected Output

```
Domains before AC-3: X=[1, 2, 3], Y=[1, 2, 3], Z=[1, 2, 3]
Arc-consistent: True
Domains after AC-3 (fixpoint): X=[1], Y=[2], Z=[3]
```

Matches the hand trace exactly – this is what “independently checkable” means.

3 • Participation Quick Check: Trace $A \leq B, B \leq C$

Domains $\{1, 2\}$ each. List the arcs processed, with any domain change, and give the fixpoint.

(A, B) : no change ($A=1 \rightarrow B \in \{1, 2\}$, $A=2 \rightarrow B=2$); (B, A) : no change; (B, C) , (C, B) : symmetric, no change – the worklist empties with **nothing reduced at all**:

$A=\{1, 2\}$, $B=\{1, 2\}$, $C=\{1, 2\}$. Arc consistency does not always prune anything – it is a *necessary* filtering step, not a solving procedure by itself.

SECTION 3

Unit 3 — Global Constraints

Local Consistency AC-3 **Global Constraints** Measuring Strength Summary

2 • Content Five Constraints, One Shared Shape

Five global constraints,
what each one propagates

AllDifferent

removes a value with
no valid distinct-
assignment match for
the whole group

Element

keeps index and target
consistent with a
fixed lookup array, in
both directions

Table

keeps only values
appearing in some
still-allowed
explicit tuple

Circuit

forbids any sub-
cycle among fewer
than all the
nodes

Cumulative

forbids
overlapping tasks
from exceeding
shared capacity

A global constraint is a single filtering rule over a whole group of variables – it can see relationships a pile of small binary constraints never gets to look at jointly.

AllDifferent: Régin's Matching-Based Filtering

- Build a bipartite graph of variables to values; a value survives only if it is in some **perfect matching** that assigns every variable a distinct value
- `model.AddAllDifferent(...)` implements exactly this – GAC, not the weaker result of $n(n-1)/2$ separate `!=` calls
- Exactly what catches Unit 1's A, B, C pigeonhole toy

Analogy

A hiring committee checking pairwise “can A and B both still get a distinct slot” can look fine for every pair, yet there may be no way to give *all* candidates distinct slots at once – only checking the whole group together catches that.

Element: A Lookup That Propagates Both Ways

Definition: Element constraint

`target == array[index]`, for a fixed array; propagates both ways between `index` and `target`.

Analogy

Looking up a price on a numbered menu board: knowing only that the price must be at least some amount already rules out every item priced too low.

Menu prices [10, 25, 40, 15]; adding `target >= 20` removes `index = 0` and `index = 3`, never fixing `index` itself.

Table: Allowed Combinations, Listed Explicitly

Definition: Table constraint

Allowed combinations given as an explicit list of tuples, not a formula; keeps, per variable, only values in some compatible tuple.

Analogy

A room/time compatibility chart from a facilities office: it just lists every allowed pair directly – useful when the combinations do not reduce to a formula.

A Table of the 6 allowed (`color1`, `color2`) pairs with `color1 != color2` filters *here* just like `!=` – but Table can express relations `!=` cannot.

Circuit and Cumulative: Tiny Toy Demos

Definition: Circuit

Forces chosen arcs to form a *single* Hamiltonian cycle over all nodes, not several disjoint sub-cycles.

A single courier's route: every stop visited exactly once, one connected loop.

Deliberately tiny instances only – 4 nodes, 3 tasks; full routing and scheduling models are Weeks 14 and 11's job.

Definition: Cumulative

Forbids any overlapping tasks from exceeding a shared resource's capacity at any instant.

A shared lab room's occupancy cap: overlapping experiments must never exceed capacity.

Code: Element Demo

```
1 model = cp_model.CpModel()
2 index = model.NewIntVar(0, len(prices) - 1, "index")
3 target = model.NewIntVar(min(prices), max(prices), "target")
4 model.AddElement(index, prices, target)
5 before = _enumerate_values(model, index)
6
7 model.Add(target >= 20)
8 after = _enumerate_values(model, index)
9
10 print(f"Element demo, menu prices={prices}")
11 print(f"index domain before target >= 20: {before}")
12 print(f"index domain after target >= 20: {after}")
```


Code: Table Demo

```
1 model = cp_model.CpModel()
2 color1 = model.NewIntVar(0, 2, "color1")
3 color2 = model.NewIntVar(0, 2, "color2")
4 model.AddAllowedAssignments([color1, color2], allowed)
5 model.Add(color1 == 0)
6 remaining = _enumerate_values(model, color2)
7
8 print(f"Table demo, allowed (color1, color2) pairs: {allowed}")
9 print(f"color1 fixed to {colours[0]}; color2 narrows to "
10       f"{[colours[v] for v in remaining]}")
```

Code: Circuit Demo

```
1 model = cp_model.CpModel()
2 arcs = [
3     (i, j, model.NewBoolVar(f"arc_{i}_{j}"))
4     for i in range(n)
5     for j in range(n)
6     if i != j
7 ]
8 model.AddCircuit(arcs)
9
10 solver = cp_model.CpSolver()
11 solver.parameters.num_workers = 1
12 solver.parameters.random_seed = 42
13 solver.Solve(model)
14 chosen = {(i, j) for i, j, lit in arcs if solver.Value(lit)}
15 cycle = _extract_cycle(chosen, n)
16 print(f"Circuit demo ({n} nodes): visiting order {cycle} -> back to {cycle[0]}")
```

Code: Cumulative Demo

```
1 model = cp_model.CpModel()
2 starts = [model.NewIntVar(0, horizon, f"start_{t}") for t in range(3)]
3 intervals = [
4     model.NewFixedSizeIntervalVar(starts[t], durations[t], f"task_{t}")
5     for t in range(3)
6 ]
7 model.AddCumulative(intervals, demands, capacity)
8
9 solver = cp_model.CpSolver()
10 solver.parameters.num_workers = 1
11 solver.parameters.random_seed = 42
12 solver.Solve(model)
13 result = [solver.Value(s) for s in starts]
14 print(f"Cumulative demo, durations={durations}, demands={demands}, "
15       f"capacity={capacity}: start times = {result}")
```

Real Output: All Four Demos

Expected Output

```
Element demo, menu prices=[10, 25, 40, 15]  
index domain before target >= 20: [0, 1, 2, 3]  
index domain after target >= 20: [1, 2]
```

Expected Output

```
Table demo, allowed (color1, color2) pairs: [(0, 1), (0, 2), (1, 0), ...]  
color1 fixed to R; color2 narrows to ['G', 'B']
```

Real Output: Circuit and Cumulative

Expected Output

Circuit demo (4 nodes): visiting order [0, 2, 1, 3] -> back to 0

Expected Output

Cumulative demo, durations=[3, 2, 2], demands=[2, 3, 2],
capacity=4: start times = [2, 0, 2]

3 • Participation Quick Check: Table vs. !=

Two variables `color1`, `color2`, domain $\{R, G, B\}$ each: (a) a `Table` listing the 6 allowed unequal-colour pairs, (b) a plain `color1 != color2`.

- Do they filter the same values here?
- Name one situation where `Table` and `!=` would *not* agree

Yes, identical here, because the allowed-tuple list was built to contain exactly what `!=` already allows; they disagree whenever the allowed combinations do not reduce to a clean symbolic rule – e.g. an explicit room/time chart where some same-looking pairs are allowed and others are not for reasons no single inequality captures.

SECTION 4

Unit 4 — Measuring Propagation Strength

Local Consistency AC-3 Global Constraints **Measuring Strength** Summary

2 • Content Same Problem, Two Models

- **Strong model:** Week 1's `nqueens.py`, unchanged – one `AllDifferent` over columns, one over each diagonal transform
- **Weak model:** the identical relation as a pairwise decomposition – a `!=` for every pair of columns and every pair of diagonal values
- Same variables, same diagonal transforms – the **only** difference is how “no two share a value” is stated

Analogy

Reading every pairwise course-clash rule separately vs. reading one master “this whole cohort’s exam times must all differ” rule – both describe the same requirement; the master rule lets a scheduler notice a conflict earlier.

Reading Solver Statistics, Not Just the Answer

- `solver.NumBranches()`, `solver.NumConflicts()`, `solver.WallTime()` – how much work the search actually did
- The empirical stand-in for “how much did propagation prune before/during search”: fewer dead values left $\Rightarrow$ fewer branches needed
- **Honest caveat:** CP-SAT’s own presolve is already good at tightening some pairwise decompositions – the measured gap can be smaller than the argument alone suggests. Report the real numbers; do not fabricate a large gap

Code: Strong vs. Weak Constraint Layers

```
1 model.AddAllDifferent(queens)
2 model.AddAllDifferent([queens[i] + i for i in range(n)])
3 model.AddAllDifferent([queens[i] - i for i in range(n)])
```

```
1 for i in range(n):
2     for j in range(i + 1, n):
3         model.Add(queens[i] != queens[j])
4         model.Add(queens[i] + i != queens[j] + j)
5         model.Add(queens[i] - i != queens[j] - j)
```

Same variables (`nqueens_variables`, carried over from Week 1) – only this constraint layer differs between the two models.

Code: Collecting One Model's Statistics

```
1 solver = cp_model.CpSolver()  
2 solver.parameters.num_workers = 1  
3 solver.parameters.random_seed = 42  
4 status = solver.Solve(model)  
5 return {  
6     "status": solver.StatusName(status),  
7     "branches": solver.NumBranches(),  
8     "conflicts": solver.NumConflicts(),  
9     "wall_time": solver.WallTime(),  
10 }
```

One call, one model – run it once for the strong model, once for the weak model.

Code: Reporting the Comparison

```
1 strong_model, _ = build_strong_model(n)
2 weak_model, _ = build_weak_model(n)
3 stats = {
4     "strong (AllDifferent)": collect_solver_stats(strong_model),
5     "weak (pairwise !=)": collect_solver_stats(weak_model),
6 }
7 print(f"N={n} propagation-strength comparison")
8 for label, s in stats.items():
9     print(
10         f"{label}: status={s['status']} branches={s['branches']} "
11         f"conflicts={s['conflicts']} wall_time={s['wall_time']:.4f}s"
12     )
13 return stats
```

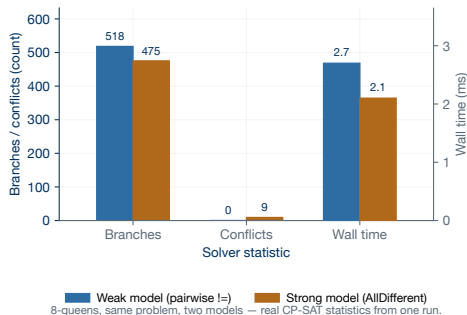
Real Output: $N = 8$

Expected Output

```
N=8 propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=475 conflicts=9
    wall_time=0.0021s
weak (pairwise !=): status=OPTIMAL branches=518 conflicts=0
    wall_time=0.0027s
```

The strong model needs fewer branches, as expected – but it also reports *more* conflicts, and takes slightly longer. Real solver output does not always point the same direction on every statistic at once.

The Proof, as a Chart



Sourced directly from `nqueens-propagation-stats.txt` – no invented numbers.

Real Output: $N = 20$ – Does the Gap Grow?

Expected Output

```
N=20 propagation-strength comparison
strong (AllDifferent): status=OPTIMAL branches=3312 conflicts=51
    wall_time=0.0090s
weak (pairwise !=): status=OPTIMAL branches=3940 conflicts=26
    wall_time=0.0201s
```

- Weak needs **more** branches and **more** wall time at both N – consistent with the Unit 1/3 argument
- Weak reports **fewer** conflicts at both N – a genuinely mixed signal, not an error to hide or explain away

3 • Participation Quick Check: Would AllDifferent Work Here?

A student proposes simplifying Week 1's map-coloring model by replacing every pairwise adjacency $\neq$ with one `AllDifferent` over *all* regions.

- Is this a valid, meaning-preserving change?

No – `AllDifferent` over all regions would force every region to differ from every other region, including non-adjacent ones. Map coloring only requires *adjacent* regions to differ. A global constraint only replaces a decomposition when the intended relation genuinely is “everyone in this group must differ from everyone else,” as it is for N-queens' columns – not whenever a shortcut merely looks similar.

SECTION 5

Summary

Local Consistency AC-3 Global Constraints Measuring Strength **Summary**

Summary

Summary

- $\text{Node} \subset \text{arc} \subset \text{GAC}$ – how much of the problem a value must be checked against before it is trusted
- AC-3: a worklist of arcs, each **Revise** re-queueing affected neighbours, until a fixpoint
- Five global constraints – **AllDifferent**, **Element**, **Table**, **Circuit**, **Cumulative** – filter a whole group jointly
- A weak vs. strong CP-SAT model of the same problem turns the argument into real, measurable solver statistics

4 • Assessment Check Yourself

- One Check Yourself question closes each unit – answers are collected at the chapter's end, not given immediately
- Node vs. arc consistency on a two-field registration form, AC-3's complexity and why it terminates, `AllDifferent` vs. pairwise `!=` on the pigeonhole case, and the map-coloring `AllDifferent` trap you just saw

5 • Follow-through Transfer — Where Else?

- Every CP-SAT call for the rest of this course runs some form of this week's consistency-checking silently, before and during search – Week 3 shows what happens once search starts consulting these same domains, and `AllDifferent`, `Cumulative` and `Circuit` return by name in Weeks 6, 11 and 14. The worklist/fixpoint pattern you traced by hand reappears, in different notation, in Week 10's LP sensitivity updates and Week 15's unsat-core extraction. This course has no running term project – instead, “same problem, two models, compare the real solver statistics” is a habit Week 6, 7 and 15 keep asking you to repeat.

In the Lab

This week's lab sheet

You classify a small CSP by consistency level, run `ac3.py` on a new toy chain and compare it against a hand trace, run all four global constraint demos, and close by reproducing this week's weak-vs-strong comparison – optionally at a larger N .

- Next week: search itself – variable and value ordering heuristics on the very same N-queens and map-coloring models
- Every **Revise** call from this week keeps firing inside that search tree – you will finally see where